

**DATABASE SYSTEMS
LABORATORY WORK
Week 2: Relational Model & Keys**

Part 1: Key Identification Exercises

Task 1.1 Relation A: Employee

Employee (EmpID, SSN, Email, Phone, Name, Department, Salary)

1. Superkeys:

- {EmpID}
- {SSN}
- {Email}
- {Phone}
- {EmpID, Name}
- {EmpID, Department}

2. Candidate Keys:

- {EmpID}
- {SSN}
- {Email}
- {Phone}

3. Primary Key:

I would choose **EmpID** as the primary key because it is unique, simple, and specifically designed to identify employees.

4. Can two employees have the same phone number?

The sample data does not contain duplicate phone numbers. However, the data shown does not explicitly state that phone numbers must always be unique. Therefore, the sample only shows that the three employees have different phone numbers.

Relation B: Course Registration

Registration (StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Primary Key:

(StudentID, CourseCode, Section, Semester, Year)

2. Why is each attribute necessary?

- **StudentID** identifies the student.
- **CourseCode** identifies the course.
- **Section** identifies the specific course section.
- **Semester** identifies the semester.
- **Year** identifies the academic year.

All these attributes are necessary because a student can take the same course in different semesters, but cannot register for the same course section in the same semester.

3. Additional Candidate Keys:

No additional candidate keys are identified from the given business rules.

Task 1.2 — Foreign Key Design

Foreign Key Relationships:

1. Student.AdvisorID → Professor.ProfID
2. Course.DepartmentCode → Department.DeptCode
3. Department.ChairID → Professor.ProfID
4. Enrollment.StudentID → Student.StudentID
5. Enrollment.CourseID → Course.CourseID

Explanation:

- AdvisorID references the professor who advises the student.
- DepartmentCode references the department that offers the course.
- ChairID references the professor who is the department chair.
- StudentID references the student in the enrollment record.
- CourseID references the course in the enrollment record.

Part 2

Task 2.1 — Hospital Management System

Entities:

- Patient — Strong entity
- Doctor — Strong entity
- Department — Strong entity

- Appointment — Strong entity
- Prescription — Strong entity
- HospitalRoom — Weak entity

Patient Attributes:

- PatientID — Primary Key
- Name
- BirthDate
- Address — Composite: Street, City, State, ZIP
- Phone — Multivalued
- Insurance

Doctor Attributes:

- DoctorID — Primary Key
- Name
- Specialization — Multivalued
- Phone — Multivalued
- OfficeLocation

Department Attributes:

- DepartmentCode — Primary Key
- Name
- Location

Appointment Attributes:

- AppointmentID — Primary Key
- DateTime
- Purpose
- Notes

Prescription Attributes:

- PrescriptionID — Primary Key
- Medication
- Dosage
- Instructions

HospitalRoom Attributes:

- RoomNumber
- DepartmentCode

• Primary Key: (DepartmentCode, RoomNumber)

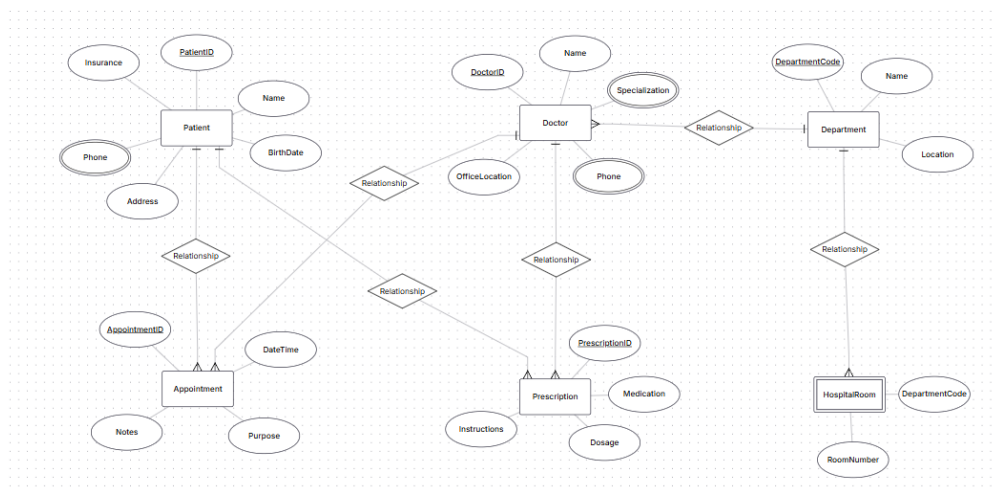
Relationships:

1. Department — Doctor: 1:N
2. Department — HospitalRoom: 1:N
3. Patient — Appointment: 1:N
4. Doctor — Appointment: 1:N
5. Patient — Prescription: 1:N
6. Doctor — Prescription: 1:N

7. Weak Entity Explanation:

HospitalRoom is a weak entity because its room number is only unique within a department. Therefore, a room is identified by the combination of DepartmentCode and RoomNumber.

8. ER Diagram:



Part 4: Normalization Workshop

Task 4.1 — Denormalized Table Analysis

Given Table:

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. Functional Dependencies:

- $\text{StudentID} \rightarrow \text{StudentName}, \text{StudentMajor}$
- $\text{ProjectID} \rightarrow \text{ProjectTitle}, \text{ProjectType}, \text{SupervisorID}$
- $\text{SupervisorID} \rightarrow \text{SupervisorName}, \text{SupervisorDept}$
- $(\text{StudentID}, \text{ProjectID}) \rightarrow \text{Role}, \text{HoursWorked}, \text{StartDate}, \text{EndDate}$

2. Redundancy and Anomalies:

Redundancy:

Student and project information is repeated for every student-project combination. Supervisor information is also repeated for every project supervised by the same supervisor.

Update anomaly:

If a supervisor's department changes, many rows must be updated. If one row is missed, inconsistent data appears.

Insert anomaly:

A new student or project cannot be added easily unless a student-project relationship also exists.

Delete anomaly:

Deleting the last student assigned to a project may also delete the only stored information about that project or supervisor.

3. 1NF:

The table is already in 1NF because all attributes contain atomic values and there are no repeating groups.

4. 2NF:

Primary Key: (StudentID, ProjectID)

Partial Dependencies:

- $\text{StudentID} \rightarrow \text{StudentName}, \text{StudentMajor}$
- $\text{ProjectID} \rightarrow \text{ProjectTitle}, \text{ProjectType}, \text{SupervisorID}$

2NF Decomposition:

STUDENT(StudentID PK, StudentName, StudentMajor)

PROJECT(ProjectID PK, ProjectTitle, ProjectType, SupervisorID)

STUDENT_PROJECT(StudentID PK/FK, ProjectID PK/FK, Role, HoursWorked, StartDate, EndDate)

5. 3NF:

Transitive Dependency:

ProjectID $\rightarrow$ SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

Final 3NF Decomposition:

STUDENT(StudentID PK, StudentName, StudentMajor)

SUPERVISOR(SupervisorID PK, SupervisorName, SupervisorDept)

PROJECT(ProjectID PK, ProjectTitle, ProjectType, SupervisorID FK)

STUDENT_PROJECT(StudentID PK/FK, ProjectID PK/FK, Role, HoursWorked, StartDate, EndDate)

Task 4.2 — Advanced Normalization

Given Table:

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1. Primary Key:

(StudentID, CourseID, InstructorID, TimeSlot, Room)

This key identifies a student's enrollment in a specific course section.

2. Functional Dependencies:

- StudentID $\rightarrow$ StudentMajor
- CourseID $\rightarrow$ CourseName
- InstructorID $\rightarrow$ InstructorName
- (TimeSlot, Room) $\rightarrow$ Building
- (CourseID, InstructorID, TimeSlot, Room) $\rightarrow$ Course section

3. BCNF Check:

The table is **not in BCNF** because some determinants are not superkeys.

4. BCNF Decomposition:

STUDENT(StudentID PK, StudentMajor)

COURSE(CourseID PK, CourseName)

INSTRUCTOR(InstructorID PK, InstructorName)

ROOM_SCHEDULE(TimeSlot, Room, Building, PK: TimeSlot, Room)

COURSE_SECTION(CourseID, InstructorID, TimeSlot, Room,

PK: CourseID, InstructorID, TimeSlot, Room)

ENROLLMENT(StudentID, CourseID, InstructorID, TimeSlot, Room, PK: StudentID, CourseID, InstructorID, TimeSlot, Room)

5. Potential Information Loss:

The decomposition preserves the information about students, courses, instructors, rooms, and enrollments. However, if the original table contains a course section identified by information not included in the given attributes, that information may not be uniquely reconstructable.

Note: The original task does not provide a separate SectionID, so the primary key above is based on the given attributes and business rules.

Part 5: Design Challenge

Task 5.1 — University Clubs

1. Entities:

- **Student** — StudentID, Name
- **Club** — ClubID, Name, Budget, AdvisorID
- **Membership** — StudentID, ClubID, JoinDate
- **Event** — EventID, ClubID, Name, DateTime, RoomID
- **Attendance** — StudentID, EventID
- **OfficerPosition** — PositionID, PositionName
- **ClubOfficer** — ClubID, StudentID, PositionID
- **Faculty** — FacultyID, Name
- **Room** — RoomID, Location
- **Expense** — ExpenseID, ClubID, Amount, Date, Description

2. Relationships:

- Student — Club: **M:N** through Membership
- Club — Event: **1:N**
- Student — Event: **M:N** through Attendance
- Club — Faculty: **N:1**
- Club — Student: **M:N** through ClubOfficer
- Club — Room: **1:N**

•Club — Expense: **1:N**